



THU
Technische
Hochschule
Ulm

HENSOLDT

Bachelor thesis at Ulm University of Applied Sciences

Department of Computer Science

Degree Program Informatik

Presented by Gabriel Zimmermann

April 2026

1. Reviewer: Prof. Dr.-Ing. Klaus Baer
2. Reviewer: Prof. Dr.-Ing. Philipp Graf

Place, Date, Signature

Diese Abschlussarbeit wurde von mir selbständig verfasst. Es wurden nur die angegebenen Quellen und Hilfsmittel verwendet. Alle wörtlichen und sinngemäßen Zitate sind in dieser Arbeit als solche kenntlich gemacht.

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references. Place, date and signature.

Place, Date, Signature

Plugin Architectures with Rust - an Analysis of Obstacles and Prospects

Gabriel Zimmermann

I. Abstract

Plugin architectures enable extensible software systems by allowing runtime loading of external modules. While Rust offers memory safety guarantees and performance characteristics ideal for plugin development, the language's lack of a stable Application Binary Interface (ABI) presents significant challenges for cross-module compatibility. This thesis examines the landscape of plugin system implementations in Rust, evaluating three primary approaches: native Rust plugins, C ABI-compatible interfaces, and Inter-Process Communication (IPC) mechanisms. Through comparative analysis of technical requirements, performance implications, and maintenance considerations, this work demonstrates that Rust's unstable ABI hampers reliable direct plugin loading across compiler versions. The findings indicate that C ABI remains the optimal choice when performance is paramount, while IPC-based architectures provide superior isolation and version compatibility at the cost of increased overhead. These results contribute practical guidance for developers designing extensible Rust applications and highlight ongoing gaps in Rust's ecosystem for safe, performant plugin architectures.

Contents

I)	Abstract	3
II)	Introduction	8
III)	Related work	9
III.A)	Foundational Concepts	9
III.A.a)	Plugin Systems	9
III.A.b)	Application Binary Interface (ABI)	9
III.A.c)	Rust Types and ABI Instability	9
III.A.d)	Dynamic Linking and Runtime Loading	10
III.B)	Existing work	11
IV)	Methods	12
IV.A)	Analysis of the requirements	12
IV.A.a)	Performance Overhead	12
IV.A.b)	Development complexities	12
IV.A.c)	Limitations	13
IV.A.d)	Interoperability	13
IV.A.e)	Safety	14
IV.B)	Potential approaches and problems	14
IV.B.a)	Unstable Rust ABI	14
IV.B.b)	abi_stable crate	14
IV.B.c)	rust-bindgen	14
IV.B.d)	cbindgen crate	15
IV.B.e)	stabby crate	15
IV.B.f)	Other Rust Bindings	15
IV.B.f.a)	pyo3 (Python bindings)	15
IV.B.f.b)	Wasm (WebAssembly)	15
IV.B.g)	Interprocess Communication (IPC)	15
IV.B.g.a)	rust_bridge crate	15
IV.B.g.b)	other ipc crates? (overview)	15
IV.C)	Selected approach and detailed solutions	16
V)	Results	17
V.A)	Validation of the overall concept	17
V.B)	Description and motivation for the test cases	17
V.B.a)	Control group: Static and Dynamic variation	17
V.B.a.a)	Performance	17
V.B.a.b)	Development complexity	18
V.B.a.c)	Limitation	18
V.B.a.d)	Interoperability	18
V.B.b)	Unstable Rust ABI	18
V.B.b.a)	Performance	18
V.B.b.b)	Development complexity	19
V.B.b.c)	Limitation	19
V.B.b.d)	Interoperability	19
V.B.c)	abi_stable (crate)	19
V.B.c.a)	Performance	19
V.B.c.b)	Development complexity	20
V.B.c.c)	Limitation	20
V.B.c.d)	Interoperability	20
V.B.d)	stabby (crate)	20
V.B.d.a)	Development complexity	20
V.B.d.b)	Limitation	20
V.B.d.c)	Interoperability	20
V.B.e)	rust_bridge (crate)	20
V.B.e.a)	Performance	20

V.B.e.b) Development complexity	22
V.B.e.c) Limitation	22
V.B.e.d) Interoperability	23
V.C) Overview and evaluation of the acquired results	23
VI) Conclusion and Future Work	24
VII) Listings	25
References	26

Abbreviations:

- ABI: Application Binary Interface
- IPC: Interprocess Communication
- RPC: Remote Procedure Call
- ZST: Zero-Sized Type
- FFI: Foreign Function Interface

II. Introduction

Modern software systems increasingly demand flexibility and extensibility. Plugin architectures have emerged as a fundamental pattern for achieving this goal, enabling applications to load additional functionality at runtime without recompilation. As highlighted by Apple's technical documentation, "Plug-in architectures are an attractive solution for developers seeking to build applications that are modular, customizable, and easily extensible" [1].

The rise of the rust language popularity has led to an increased interest in using Rust for plugin development. Rust has established itself as a powerful systems programming language that offers memory safety guarantees and performance characteristics. However, the language's design philosophy prioritizes compile-time guarantees over binary stability, resulting in an intentionally unstable Application Binary Interface (ABI). Small changes in your written code, can impact the resulting binary in a way that it is not compatible with the previous version. With that being said, we can circumvent this problem by using other approaches.

One popular approach is to use C ABI-compatible interfaces, which can provide a stable interface for plugins while still allowing developers to write their plugins in Rust. The C ABI has been the backbone for many plugin systems in various programming languages, including the Rust ecosystem. But this approach requires the careful handling of unsafe code and can lead to some performance overhead due to the need for FFI (Foreign Function Interface) calls. In addition, this can lead to the loss of Rust's safety guarantees, as the C ABI does not provide the same level of safety as Rust's native interfaces.

Another approach is to use Inter-Process Communication (IPC) mechanisms, which can provide a more flexible and scalable architecture for plugin systems. However, this approach can also introduce additional complexity and overhead, as it requires the management of separate processes and communication channels. This will impact the performance of the plugin system, as IPC can introduce latency and reduce the overall efficiency of the system. However, it can provide better isolation and security for plugins, as they run in separate processes.

In this thesis, we are interested in understanding the advantages and disadvantages of the stated approaches and what Crates are offered within the Rust Ecosystem. Later on, we will evaluate the different approaches based on the performance overhead using a simple benchmark, the resulting development complexity, limitations and interoperability.

III. Related work

When developing a plugin system, one needs to find a way to ensure that the host and the plugin can communicate with each other. This is typically done through a well-defined Application Binary Interface (ABI) or through Inter-Process Communication (IPC) mechanisms.

A. Foundational Concepts

To contextualize the challenges of plugin systems in Rust, it is essential to establish precise definitions of the core architectural components involved: what is a plugin system, what is an ABI, how stable the Rust Type System is for the ABI, and what are dynamic linking mechanisms.

a) Plugin Systems:

A plugin is a “computer software that adds new functions to a host program without altering the host program itself” [2]. A plugin system is a software architecture pattern that allows a host application to be extended by third-party modules (plugins) without modifying the host's source code or recompiling the entire system. Key characteristics include extensibility, modularity, and dynamic loading capabilities.

- **Extensibility:** The ability to add new functionality post-deployment.
- **Modularity:** Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading:** Plugins are typically loaded at runtime rather than linked statically at compile time.

When writing a Plugin systems, developers generally follow one of two patterns:

- **Static Linking:** Plugins are compiled into the host binary. This offers performance benefits but lacks runtime flexibility.
- **Dynamic Linking:** Plugins are separate binary files loaded into the host process memory space at runtime.

In this thesis, the focus is on dynamic linking, as it necessitates strict ABI compatibility or robust isolation mechanisms.

The primary challenge in Rust plugin systems is ensuring that the host and the plugin, potentially compiled with different compiler versions or optimization flags, agree on the binary representation of data and function calls.

b) Application Binary Interface (ABI):

While the Application Programming Interface (API) defines a “sets of standardized requests that allow different computer programs to communicate with each other” [3] like function names and argument types, the Application Binary Interface (ABI) operates at a lower level, governing the interaction between compiled machine code.

This covers aspects such as:

- **Calling Conventions:** How function arguments are passed (registers vs. stack), how return values are delivered, and who is responsible for cleaning up the stack (caller vs. callee).
- **Data Layout:** The memory alignment, padding, and byte-ordering (endianness) of data structures.
- **Name Mangling:** The algorithm used by compilers to encode function names (including type information) into symbol names for the linker.
- **Exception Handling:** The mechanism for propagating errors or exceptions across module boundaries.

Relevance to Rust: In the context of Rust, the lack of a standardized ABI is a deliberate design choice. As noted by the Rust compiler team, the ABI is “deliberately unstable” to allow the compiler to optimize memory layouts and calling conventions aggressively without being constrained by backward compatibility requirements (Rust Internals, 2020). This contrasts with the C ABI, which is strictly defined and stable, serving as the de facto standard for binary interoperability.

c) Rust Types and ABI Instability:

[4]

Product types (or structs): is it ordered? is it aligned?

C is always aligned and source-ordered.

Rust fields also aligned, but the order is minimized for memory usage.

Rust can enforce this by using `repr(C)`, but this is not the default, and the compiler can change the layout for optimization purposes. This means that without explicit annotations, the memory layout of a Rust struct is not guaranteed to be stable across different compiler versions or optimization levels, which can lead to binary incompatibility when used in a plugin system.

Sum types (or enums): how to distinguish the variants? where is the data?

Not supported by C. Can be done using a tagged union, but you need to give each type a specific number (usually an enum).

Rust may be put in a niche (like common padding or using type's forbidden value).

Example with `Cow<'a, str>`: Can waste 7 bytes for padding on a 64-bit architecture.

Rust guarantees that the layout of a sum type is not clashing with the layout of a product type.

Unit types (`()`): are they zero-sized? what about references?

Doesn't exist in C as zero-sized type. ZSTs are undefined behavior.

As Pierre states, for a stable ABI all linkers must agree on an ABI for each symbol [4] (double check, maybe I need to link the talk instead...).

Note stable due to changes in compiler version, optimization level, target architecture, using `-Z randomize-layout` and so on. This is a fundamental problem for plugin systems in Rust, as the host and the plugin may be compiled with different versions of the compiler or different optimization flags, leading to binary incompatibility.

A few examples as to why the Rust ABI is not stable yet can be found in niche optimizations: enums with multiple data variant optimizations (1.65), field ordering optimizations (1.67), `Cow<str>` regression (1.70), and more.

d) *Dynamic Linking and Runtime Loading*:

Dynamic Linking is the process of resolving references to external libraries or modules at runtime, rather than at compile time (link time).

Mechanism: In Unix-like systems, this is achieved through the `dlopen` (dynamic open), `dlsym` (dynamic symbol lookup), and `dlclose` (dynamic close) system calls. In Windows, the equivalents are `LoadLibrary`, `GetProcAddress`, and `FreeLibrary`.

Process Flow:

- **Discovery**: The host application locates the plugin binary file (e.g., `.so`, `.dll`, `.dylib`).
- **Loading**: The operating system maps the plugin's code and data segments into the host's virtual memory space.
- **Resolution**: The host resolves symbol addresses (function pointers) within the loaded plugin.
- **Execution**: The host invokes the plugin's functions via the resolved pointers.

Challenges in Rust: Dynamic linking in Rust is complicated by the language's ownership model and panic handling. Since Rust does not guarantee a stable ABI, a plugin compiled with Rust version 1.70 might use a different struct layout or calling convention than a host compiled with version 1.75. Furthermore, if a plugin panics, the unwinding behavior must be contained; otherwise, it can lead to undefined behavior or crashes in the host process. As described in implementation guides, "panics cannot cross the FFI boundary" without explicit handling, such as wrapping function bodies in `catch_unwind` or using ABI-stable libraries like `abi_stable` [5].

No comprehensive comparison between possible solutions for plugin systems in Rust has been done yet. There are some simple comparisons, but they don't cover the full range of options and trade-offs.

B. Existing work

As of 2026, the Rust ecosystem remains in a state of intentional ABI instability, a condition that has persisted despite years of community advocacy and formal proposals. Many proposals were held, such as RFC 1675 [6], RFC 600 [7] and discussion in the Rust Programming Language Forums, like “A Stable Modular ABI for Rust” [8], which proposes to enable runtime loading of Rust libraries without C interop. However, the proposal was ultimately deferred, with the compiler team citing the high cost of freezing internal representations that are critical for future optimizations, such as constant evaluation and monomorphization strategies [6]. Instead of a native solution, the community has converged on pragmatic workarounds, where the usage of the C ABI is the only reliable standard for binary interoperability, forcing developers to adopt verbose `extern "C"` interfaces or rely on third-party crates like `abi_stable` to simulate stability through `#[repr(C)]` wrappers and runtime checks (NullDeref, 2023; Arroyo Blog, 2023). While experimental efforts continue to explore WebAssembly as a language-agnostic ABI alternative, no native Rust-to-Rust stable ABI has been implemented, leaving plugin developers to navigate a fragmented landscape of manual FFI management and abstraction layers.

TODO: Fix links with the concrete answer of the dev team...

Currently only rust std library and ferrocene compiler verified. Therefore, most code that needs to be verified is already limited and the solutions that can be used for Plugins are limited. Not the focus of this thesis.

IV. Methods

A. Analysis of the requirements

Goal of the research was to analyze the following points:

- Performance Overhead IV.A.a
- Development Complexities IV.A.b
- Limitations IV.A.c
- Safety IV.A.e
- Interoperability IV.A.d

a) Performance Overhead:

When measuring our plugins, we expect that the execution time of the plugin's internal functionality to be relatively similar across different approaches. However, the setup time for each approach is expected to vary, with approaches that require more complex initialization (e.g. loading a dynamic library, setting up IPC communication, serialization) are more likely to require more time.

We expect that the IPC approach will have higher performance overhead compared to the other approaches due to the need for serialization and deserialization of data, as well as the communication overhead between processes. This is because IPC typically involves more complex interactions between processes, which can introduce additional latency compared to approaches that operate within a single process. We expect the measurements for the Rust ABI and C ABI approaches to be more similar to each other, as they both involve loading and executing code within the same process, albeit with different calling conventions and potential overhead from FFI in the case of the C ABI approach.

To measure the results, we are using the *criterion* and *gungraun* crate for our benchmarks. The results of these will be separated into setup time and function execution time, which allows us to analyze the overhead of each approach in more detail.

As noted by the *criterion* documentation, the *criterion* crate is a statics-driven micro-benchmarking library, which aims to provide strong statistical confidence in detecting and estimating the size of performance improvements and regressions [9]. As stated by the *gungraun* documentation, "gungraun is a one-shot benchmarking harness and framework which uses Valgrind's Callgrind, Cachegrind, and DHAT to provide extremely accurate and consistent measurements of Rust code, making it perfectly suited to run in environments like a CI" [10].

We hope by combining both of these tools, we can get a comprehensive understanding of the performance characteristics of each approach, and how they compare to each other in terms of setup time and function execution time.

In Rust benchmarking, wrapping inputs and intermediate values in `black_box` is essential to prevent the compiler from optimizing away the code you intend to measure. Because Rust's optimizer is aggressive, it may detect that a function's result is unused or that an input is constant, leading it to eliminate the entire computation or to remove it out of a loop, resulting in artificially low (and inaccurate) timing data. The `std::hint::black_box` function acts as an opaque barrier, signaling to the compiler that the value is unknown and must be treated as having side effects, thereby forcing it to execute the code exactly as written within the benchmark. [11] Each benchmark will be run with and without `black_box` to compare the results and understand the impact of compiler optimizations on our measurements.

b) Development complexities:

When using external crates, they usually come with their own learning curve and documentation. This can add to the development time, especially if the crate is not well-documented or has a complex API. Additionally, integrating external crates into an existing codebase can sometimes lead to compatibility issues or require significant refactoring. Quantifying this is difficult, as it depends on the developer's experience, familiarity with the crate, and the specific requirements of the project. We have decided to instead focus

on a qualitative analysis of the development complexities, based on the given available data by the crates maintainers and our experience during the implementation of each approach. This will include factors such as the ease of use of the crate's API, the quality of its documentation, and any challenges faced during integration. After summarizing these, we will

We expect that the dynamic library loading approaches (Rust ABI and C ABI) may have higher development complexities compared to the IPC approach, due to the need to manage dynamic libraries, handle FFI (in the case of C ABI), and ensure compatibility between the plugin and host application. The IPC approach may have lower development complexities in terms of integration, as it allows for more decoupled communication between processes, but it may require more complex software design to handle serialization and inter-process communication effectively.

We will evaluate each approach with a ++, + or 0 based on the following criteria:

- ++: The approach is straightforward to implement, with a clear and well-documented API, and requires minimal additional setup or configuration. The crate offers good documentation and examples, making it easy for developers to understand and use effectively.
- +: The approach is generally manageable to implement, but may require some additional setup or configuration. The crate has decent documentation, but may have some gaps or areas that are not well-explained, which could lead to some challenges during development.
- 0: The approach is neutral in terms of complexity, with neither significant advantages nor disadvantages in terms of implementation effort or documentation quality.
- -: The approach is complex to implement, with a steep learning curve, a poorly documented API, or significant additional setup or configuration required. Alternatively, the approach may have significant disadvantages for the developers.

c) *Limitations:*

Each approach may come with some limitations for the developers. In general, we want to see if an approach may require more complex software design, which could be a limitation for developers who are not familiar with this type of architecture.

Some Rust Types cannot be easily represented in C, which can lead to limitations in the C ABI approach. We also expect that the IPC approach may have limitations in terms of the types of data that can be easily serialized and deserialized, which could impact the flexibility of this approach. Typically, pointers and references are difficult to represent in C, while IPC might even struggle with more complex data structures that require custom serialization logic.

As the ABI solutions have different limitations Because we are also working with different ABI solutions, there can be complexities in regards of their restrictions and limitations.

In some cases, Limitations may overlap with Section IV.A.b, as certain limitations may require additional development effort to work around or mitigate. For example, if a particular approach has limitations in terms of the types of data that can be easily represented or serialized, this may require additional development effort to implement custom serialization logic or to design the plugin architecture in a way that avoids these limitations.

d) *Interoperability:*

We perform a qualitative assessment of the interoperability of each approach based on the available documentation. In our sample code, we only test the interoperability of the plugin system from a Rust Plugin to a Rust Host Application.

We will rate each approach with a ++, +, 0 based on the following criteria:

- ++: The approach provides seamless interoperability with other libraries and tools, allowing for easy integration and compatibility with a wide range of use cases.
- +: The approach has good interoperability with other libraries and tools, but may require some additional configuration or workarounds to achieve full compatibility.
- 0: The approach has limited interoperability with other libraries and tools, making it difficult to integrate with existing codebases or to use in a wide range of scenarios.

In some cases, the approach may be completely incompatible with certain libraries or tools, which can significantly limit its usefulness in real-world applications.

e) *Safety:*

The Rust Language is designed with safety in mind, and it provides strong guarantees against common programming errors such as null pointer dereferences, buffer overflows, and data races. Typically, errors occur through the careless usage of unwinding. [12]

Yet, this does not result in formal verifications, like DO-178C or IEC 61508, which are often required in safety-critical domains. However, the Rust ecosystem has been making strides in this area, with efforts to achieve certifications for certain subsets of the Rust language and libraries, such as the recent achievement of IEC 61508 (SIL 2) certification for a subset of the Rust core library by Ferrous Systems [13], [14].

From this, we can infer that while Rust provides strong safety guarantees, the level of safety can depend on the specific approach used for plugin integration. For example, approaches that involve unsafe code (such as FFI in the C ABI approach) may require more careful handling to ensure safety, while approaches that operate entirely within safe Rust (such as the Rust ABI approach) may provide stronger safety guarantees by default. The IPC approach may also have its own safety considerations, such as ensuring that data is properly validated and that communication channels are secure.

For this thesis, we do not consider it appropriate to perform a quantitative or qualitative analysis of the safety of each approach, as this would require a detailed examination of the specific implementation and usage patterns, which may be beyond the scope of this research.

B. *Potential approaches and problems*

When developing a plugin system in Rust, there are several potential approaches that can be taken, each with its own set of advantages and disadvantages. In this section we will introduce different crates and give an overview of their functionality. We will have a closer look at the chosen approach in the next section Section IV.C.

a) *Unstable Rust ABI:*

It is possible to load other Rust Libraries as plugins, using the Rust ABI. However, this approach is complicated by the fact that the Rust ABI is not stable.

b) *abi_stable crate:*

The `abi_stable` crate enables stable, version-independent interoperability between Rust libraries and executables by generating a custom, C-like ABI that bypasses Rust's unstable internal layout guarantees, allowing dynamically linked shared objects to communicate safely even when compiled with different Rust compiler versions or feature flags. It achieves this through a macro-heavy approach (`#[abi]`) that enforces strict trait bounds and generates wrapper code to handle memory layout, vtables, and function pointers, effectively creating a "Rust FFI" that is safer than raw extern "C" for complex types. However, this stability comes with significant trade-offs: the generated code introduces non-trivial runtime overhead due to extra indirection and dynamic dispatch, the API is verbose and requires extensive boilerplate that can obscure the underlying logic, and it currently lacks support for many modern Rust features like `async/await` or generic specialization, making it less suitable for performance-critical paths or projects prioritizing minimal binary size compared to simpler extern "C" or `cxx` solutions.

c) *rust-bindgen:*

The `bindgen` crate automates the creation of Rust FFI bindings by parsing C or C++ header files using Clang to generate Rust extern blocks and struct definitions that mirror the original C API, effectively eliminating the manual, error-prone process of translating C types into Rust equivalents. It operates by invoking Clang to analyze the header syntax tree, resolving macros and complex type definitions, and outputting a Rust source file that can be integrated into a build script (`build.rs`) to ensure bindings stay synchronized with header changes. However, this automation introduces significant downsides: the generated code is often verbose, unreadable, and difficult to maintain manually, leading to large

binary sizes; it tightly couples the Rust build process to the presence of a compatible Clang installation and the exact C headers, complicating cross-compilation and CI pipelines; and because it exposes the raw C API directly, it frequently forces developers to wrap the generated unsafe code in safe abstractions to prevent memory safety issues, adding an extra layer of development overhead despite the initial automation.

d) *cbindgen crate*:

The cbindgen crate facilitates Rust-to-C interoperability by scanning Rust source code annotated with specific attributes (like `#[no_mangle]` and `repr(C)`) to automatically generate C header files (.h) and corresponding documentation, allowing C or C++ projects to consume Rust libraries as if they were native C code without needing to parse Rust syntax directly. It works by traversing the Rust AST to extract public functions, structs, and enums, translating Rust types into their C equivalents (e.g., mapping `u32` to `uint32_t` and `Option<T>` to a union or pointer pattern) and handling complex scenarios like callbacks and string lifetimes through configurable build scripts. However, its effectiveness is limited by the requirement that Rust code must strictly adhere to C-compatible layouts and calling conventions, often necessitating manual refactoring of idiomatic Rust code to fit these constraints; additionally, it struggles with advanced Rust features like generics, traits, and complex lifetimes which have no direct C equivalent, forcing developers to write verbose wrapper functions or accept that the generated headers may be incomplete or require significant manual patching to be usable in a C environment.

e) *stabby crate*:

f) *Other Rust Bindings*:

pyo3 (Python bindings):

The pyo3 crate enables seamless bidirectional interoperability between Rust and Python by leveraging the Python C API to expose Rust functions, structs, and enums as native Python objects while allowing Rust code to call Python functions and manipulate Python objects with type safety. It operates through a macro-driven system where the `#[pyclass]` and `#[pymethods]` attributes generate the necessary C bindings and glue code, automatically handling reference counting, memory management, and the conversion of Rust types (like `Vec` or `String`) into their Python equivalents (like `list` or `str`) via the `IntoPy` and `FromPyObject` traits. By utilizing the Global Interpreter Lock (GIL) guard to ensure thread safety during Python interactions and providing a high-level API that abstracts away the complexities of the underlying C API, pyo3 allows developers to write performance-critical Python extensions in Rust that feel and behave like idiomatic Python code, all while maintaining the safety guarantees of the Rust compiler.

Wasm (WebAssembly):

Using WebAssembly with Rust involves compiling Rust code to the `wasm32-unknown-unknown` target, a process streamlined by tools like `wasm-pack` and `wasm-bindgen` that bridge the gap between Rust's memory model and JavaScript's environment. By annotating Rust functions with `wasm_bindgen`, developers can automatically generate the necessary glue code to export Rust functions to JavaScript and import JS functions into Rust, handling complex type conversions (such as mapping Rust Strings to JS strings and Vectors to typed arrays) and managing memory ownership across the boundary. This workflow typically starts by installing the `wasm32` target via `rustup`, writing Rust logic with the `wasm-bindgen` crate, running `wasm-pack build` to generate a `.wasm` binary and corresponding JavaScript/TypeScript bindings, and finally loading the resulting module in a browser or Node.js environment to execute high-performance, sandboxed code that interacts seamlessly with the host application.

g) *Interprocess Communication (IPC)*:

different structure. communication over serialization

positive: interoperability, less complexity during development

negative: performance overhead, more complex software design

rust_bridge crate:

other ipc crates? (overview):

C. Selected approach and detailed solutions

When choosing the approach for further analysis, we decided to take a closer look at the **Unstable Rust ABI**, the **abi_stable** crate, the **stabby** crate and on the **rust_bridge** crate. We choose those, because they represent a good variety of approaches to plugin systems in Rust, ranging from direct ABI manipulation to IPC-based communication.

The Unstable Rust ABI approach is interesting because it represents the most direct way to load Rust plugins, but it also comes with significant challenges due to the instability of the Rust ABI.

The **abi_stable** crate is interesting because it provides a solution to the instability of the Rust ABI, but it also comes with its own set of trade-offs in terms of performance and complexity. The **stabby** crate is interesting because it has a more fundamental approach (NOT HAPPY WITH THIS PHRASING!!) to plugin systems in Rust, and it also has a focus on safety and ease of use. Finally, the **rust_bridge** crate is interesting because it represents an IPC-based approach to plugin systems in Rust, which has its own set of advantages and disadvantages compared to the other approaches.

WebAssembly (Wasm) is not covered in this paper.

C ABI: **abi_stable** and **stabby**

Rust: ... well unstable

IPC: **rust_bridge**

V. Results

A. Validation of the overall concept

All our plugins have to implement the following trait:

```
trait Fuser {
  fn fuse(&self, data: &[SensorData]) -> Result<SensorData,
Box<dyn std::error::Error>>;
}
```

Listing 1. Fuser trait definition

The `Fuser` trait defines a single method, `fuse(...)`, which takes a slice of `SensorData` and returns a fused `SensorData` or an error.

```
struct SensorData {
  temperature: f64,
  deviation: f64,
}
```

Listing 2. SensorData struct definition

In our benchmark, a `SensorData` represents a tuple of temperate and deviation. From a practical sense, many temperature sensors have a small margin of error (deviation), which can be used to calculate a more accurate temperature reading by fusing the data from multiple sensors.

The implementation of the `fuse(...)` method will be the same for all plugins, as we want to focus on the performance of the different approaches, and not on the implementation of the plugin itself. The only difference will be the way we load and execute the plugin, which will be the focus of our evaluation. In some cases, we needed to make some adjustments to the function parameters and return types to fit the requirements of the different approaches. These differences will be explained in the respective sections.

As a control group, we ran the same benchmark using a static and dynamic variations first, without the use of any Plugin logic, which evaluates to `AverageFuser` and `Box<dyn Fuser>` respectively.

B. Description and motivation for the test cases

a) Control group: Static and Dynamic variation: Performance:

First, we want to establish a baseline for the performance of our benchmark by running it without any plugin logic. This will allow us to understand the overhead introduced by the plugin system and to compare the results of the different approaches against this baseline.

TABLE I
Criterion benchmark: Static variation

Test Case	Standard	Blackboxed
Creation	243.18 ps	245.37 ps
Single Fuse	1.2422 ns	2.3027 ns
Multi Fuse	856.15 μ s	865.71 μ s

TABLE II
Criterion benchmark: Dynamic variation

Test Case	Standard	Blackboxed
Creation	242.86 ps	249.83 ps
Single Fuse	1.2440 ns	2.3058 ns
Multi Fuse	859.60 μ s	865.63 μ s

When comparing table Table I and table Table II, we can see that the results are very similar, which indicates that the difference between static and dynamic variations does not have a significant impact on the performance of the benchmark. This is expected, as the only difference is how the `■` trait is implemented and called.

TABLE III
Gungraun benchmark: Static variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1	1	36	36
Single Fuse	43	89	94	333
Multi Fuse	92	4,500,136	225	15,000,936

TABLE IV
Gungraun benchmark: Dynamic variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1	21	2	140
Single Fuse	99	108	320	371
Multi Fuse	4,500,146	4,500,155	15,001,017	15,001,102

The estimated cycles of the table Table III and table Table IV show that the final results are very similar.

Development complexity:

We are sticking to the standard Rust development process, which is well-documented and widely used in the Rust community. This means that we are using the standard Rust toolchain, including Cargo for package management and build automation, and we are following the standard Rust coding conventions and best practices. This approach allows us to leverage the existing Rust ecosystem and resources, and it provides a familiar development experience for Rust developers.

As this is the control group, this represent the state of rust and therefore we are assigning a rating of ++.

Limitation:

Our control group does not have any **limitations** on a technical level, as we are not introducing any dynamic loading or plugin logic here. We are using the base library of Rust and all of the features this language provides. This is standard rust and therefore the limitations are the same as for any other Rust project.

While it is not possible to introduce dynamic plugins in this control group, we can still put this as optimal showcase of minimal limitations that a plugin system might have - therefore we rate this as: ++.

Interoperability:

In this control group, **interoperability** is not an option, as we cannot introduce dynamic plugins here. Using the metrics stated in Section IV.A.d, we can evaluate the control group as follows: 0.

b) Unstable Rust ABI:

Performance:

Running the benchmark with the Unstable Rust ABI approach, we can see that the performance is significantly worse compared to the control group, which is expected due to the overhead of communication via FFI.

TABLE V
Criterion benchmark: Unstable Rust ABI variation

Test Case	Standard	Blackboxed
Creation	218.24 μ s	217.24 μ s
Single Fuse	8.1845 ns	7.4945 ns
Multi Fuse	0.998 ms	1.0009 ms

TABLE VI
Gungraun benchmark: Unstable Rust ABI variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	1,090,531	1,090,534	1,637,886	1,637,923
Single Fuse	182	191	553	604
Multi Fuse	10,500,230	10,500,239	21,001,579	21,001,630

Development complexity:

For the Unstable Rust ABI approach, we are using the standard Rust development process, which is well-documented and widely used in the Rust community. However, due to the unstable nature of the Rust ABI, we need to be careful when updating our rust compiler version. This will essentially lock you in place for the duration of your project. This risk might be okay on a small scale, but as soon as your software or library needs to grow and incorporate other developers or libraries, it is not acceptable to use.

Therefore, we are rating the development complexity of this approach as -. We see this restriction alone to a specific Rust compiler version as a significant disadvantage.

Limitation:

The limitations of the Unstable Rust ABI approach are similar to the development complexity. You are not able to update your compiler version and therefore working with other libraries (which might require a newer version of the compiler) is restricted.

Interoperability:

There is no interoperability with the Unstable Rust ABI approach, as the plugin and the host application must be compiled with the same version of the Rust compiler to ensure compatibility. This means that you cannot use plugins compiled with a different version of Rust, which significantly limits the flexibility and usability of this approach. Other languages are also not able to understand the Rust ABI. While there are some similarities between the C and Rust ABI, the Rust compiler is using it's LLM to optimize parts of the code.

Therefore, we are rating the interoperability of this approach as 0.

c) abi_stable (crate):

Performance:

TABLE VII
Criterion benchmark: Stable Rust ABI variation

Test Case	Standard	Blackboxed
Creation	29.745 ns	29.751 ns
Single Fuse	8.8004 ns	9.2904 ns
Multi Fuse	1.0699 ms	1.0813 ms

TABLE VIII
Gungraun benchmark: Stable Rust ABI variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	675,743	675,632	1,130,034	1,129,601
Single Fuse	200	209	740	791
Multi Fuse	10,500,248	10,500,257	21,001,912	21,001,963

Development complexity:

The `abi_stable` crate provides a stable ABI for Rust, which allows developers to create plugins that can be loaded at runtime without worrying about compatibility issues. However, using this crate requires some additional setup and configuration compared to the standard Rust development process. Developers need to define their plugin interfaces using the `abi_stable` API, which may require some learning curve and additional effort to understand and use effectively.

The documentation for this is decent, but there are some gaps and areas that are not well-explained, without diving into the code itself.

In addition, the example implementation shows a very good scenario, but it doesn't explain very well, why certain decision were made.

Otherwise, the implementation requires a high amount of boilerplate code, which can be a significant barrier for developers who are not familiar with this type of architecture.

Therefore, we rate this is as approach as +, as it is generally manageable to implement, but requires some additional setup and configuration.

Limitation:

Interoperability:

d) *stabby (crate):*

TABLE IX
Criterion benchmark: stabby variation

Test Case	Standard	Blackboxed
Creation	81.095 μ s	75.139 μ s
Single Fuse	7.3371 ns	5.8786ns
Multi Fuse	933.62 μ s	907.15 μ s

TABLE X
Gungraun benchmark: stabby variation

Test Case	Instructions	Inst. (Blackboxed)	Estimated Cycles	Est. Cyc. (Blackboxed)
Creation	73,983	73,985	133,412	133,423
Single Fuse	90	95	337	380
Multi Fuse	337	380	15,000,970	15,001,013

Development complexity:

Limitation:

Interoperability:

e) *rust_bridge (crate):*

As this crate allows to differentiate between JSON and binary communication, we will evaluate both of these transport layers separately, as they have different performance characteristics and development complexities.

Performance:

Due to an `NullHandle` error during the plugin creation itself with criterion, we were not able to run the whole criterion benchmark for the `rust_bridge` crate. It appears that there

is some issue when creating the plugin itself, which seems to be related to the reading of the plugin file. Until the end of the deadline for this thesis, a solution was not found.

For the creation of the plugin, a total of 104,733,470 instructions were required, which results in an estimated 126,094,247 cycles. Using a `blackbox` version, shows that the creation of the plugin requires 104,733,497 instructions and 126,094,318 cycles, which is very similar to the standard version, and therefore indicates that the creation of the plugin is not affected by the optimizations of the compiler.

The benchmark follow the same structure that is required for communicating with the `rust_bridge` crate. First, we create a request header (json and binary; single and multiple) and then we send it to the plugin, which will process the request by performing the fusion (single or multiple) and return a response. The response has to be additionally parsed in the binary version only. Additionally, the response has the same structure regardless of input size Listing 1.

TABLE XI
Criterion benchmark: `rust_bridge` (single `SensorData`)

Test Case	JSON	JSON (Blackboxed)	Binary	Binary (Blackboxed)
Request Single	6.9305 ns	9.6154 ns	10.266 ns	10.846ns
Fuse Single	2.7541 μ s	7.7651 μ s	127.21 ns	127.45 ns
Response	-	-	2.5295ns	6.3095ns
Total (Single)	2.7610 μ s	7.7747 μ s	139.005 ns	144.605 ns

Looking at the required amount of time for the fusion (which includes the transport of the data), we can clearly see, that this is a heavy process in the JSON version, which is expected due to the overhead of parsing and serializing JSON data. In contrast, the binary version is much faster, which can be attributed to the fact that the binary version does not require parsing and serializing of JSON data. The request header creation is very fast in both versions, which indicates that the communication with the plugin is not affected by the optimizations of the compiler. The response parsing is also very fast in both versions, which indicates that the communication with the plugin is not affected by the optimizations of the compiler.

TABLE XII
Criterion benchmark: `rust_bridge` (multiple `SensorData`)

Test Case	JSON	JSON (Blackboxed)	Binary	Binary (Blackboxed)
Request Multiple	40.045 ns	965.32 μ s	969.30 μ s	974.90 μ s
Fuse Multiple	460.39 ms	455.40 ms	1.0599 ms	1.0917 ms
Response	-	-	2.5295ns	6.3095ns
Total (Multiple)	460.43 ms	455.40 ms	1.0625 ms	1.0989 ms

When taking a look at the blackboxed values, we can see that the optimizations of the compiler doesn't have a big impact on the results, which indicates that the results are not affected by any optimizations of the compiler. The fusion process is the most time-consuming part of the benchmark, which is expected due to the fact that it involves processing a large amount of data and performing complex calculations. The request header creation in the JSON version seems to be able to have some optimizations.

Comparing the results to the benchmarking results of the `rust_bridge` crate, we can see that the results are similar, which indicates that the performance of the `rust_bridge` crate is consistent with the performance of the control group. This is a good indication that the `rust_bridge` crate is performing as expected and that the results are not affected by any optimizations of the compiler. [15]

TABLE XIII
Gungraun benchmark: Instruction Count for rust_bridge

Test Case	JSON Instru- tions	JSON Inst. (Blackboxed)	Binary Instru- tions	Binary Inst. (Blackboxed)
Request Single	43	56	198	202
Request Multiple	92	105	75,382,326	75,382,330
Fuse Single	60,439	60,456	36,793	36,804
Fuse Multiple	4,518,965,517	4,519,030,617	10,536,920	10,536,931
Response	-	-	135	143

TABLE XIV
Gungraun benchmark: Estimated Cycles for rust_bridge

Test Case	JSON Instru- tions	JSON Inst. (Blackboxed)	Binary Instru- tions	Binary Inst. (Blackboxed)
Request Single	94	185	459	501
Request Multiple	225	350	155,053,204	155,053,246
Fuse Single	108,962	109,185	61,448	61,584
Fuse Multiple	6,283,037,647	6,283,110,964	21,097,406	21,097,668
Response	-	-	457	581

The instruction count show that the JSON version does require significantly less instructions compared to the binary version, but in the resulting estimated cycles, we can see that the JSON version requires significantly more cycles compared to the binary version, which indicates that the instructions in the JSON version are more complex and require more processing power compared to the binary version. This is expected due to the overhead of parsing and serializing JSON data, which can be computationally expensive.

Here it also confirms that the optimizations of the compiler doesn't have a big impact on the results, which indicates that the results are not affected by any optimizations of the compiler.

In summary, the performance of the binary transport layer is on par with the control group, while the performance of the JSON transport layer is significantly worse compared to the control group.

Development complexity:

Overall, the implementation for the rust_bridge crate is rather mixed. While the serialization and deserialization for the JSON data is straightforward and well-documented, the binary communication with the plugin is more complex and requires a deeper understanding of the data layout of your structs. That being said, the example provided for the binary approach can be followed step by step, which makes it relatively easy to implement.

The documentation of the crate is very good with extensive showcases of diagrams and examples. The technical documentation features diagrams for it's architecture, describes the lifecycle of a plugin, how the request and response flow works, and a lot more.

We rate the development complexity of the JSON transport layer as ++ due to it's simplicity, while the binary transport layer is rated as + due to the additional complexity of handling binary data and ensuring that the data layout is correct.

Limitation:

As stated by the documentation of the crate, the current limitations are related to the reloading of plugins and the usage of multiple instances. [16] Reloading of plugins is technically already possible, but due to it's inherently fragile nature (global state, background threads, third-party libraries side effects) it is currently not recommended to use this feature. Instead it is recommended to restart a process instead of a dynamic reload. For the multiple instances, the crate does support this feature, but the logging and tracing

infrastructure is not designed to handle multiple instances, which can lead to some issues when using this feature. Other limitations are related to the JSON transportation layer. There, the usage of pointers and references is not supported, which can limit the types of data that can be easily serialized and deserialized.

In total, we rate the limitations of the JSON transport layer and the binary transport layer as +. The limitations are there and need to be considered when designing a large and efficient plugin system, but for most other use cases, these limitations can be easily worked around.

Interoperability:

A plugin is exported in the form of a rbp-file. Using this file (and the corresponding crates bindings), this plugin can be used in any Rust application, but also other languages like Python, Java, Ruby and more. Due to the serialization to a JSON, it is generally very flexible. The binary layer needs a bit more work, but can be used across the language boundaries as well, as long as the data layout is correct and the corresponding bindings are implemented.

We give this crate a rating of ++ for interoperability, as it allows for a wide range of use cases and can be used in various programming languages, which makes it a very flexible solution for plugin development.

C. Overview and evaluation of the acquired results

TABLE XV
Summary of resultsnNote: Creation for rust_bridge only includes the creation of the request header

Crate	Performance (Multi Fuse)	Development Complexity	Limitation	Interoperability
native	0.856 ms	++	++	0
unstable_abi	0.998 ms			
stable_abi	1.0699 ms			
stabby	0.934 ms			
rust_bridge (JSON)	460.43 ms			
rust_bridge (Bi- nary)	1.063 ms			

VI. Conclusion and Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

VII. Listings

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensiva et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

References

- [1] A. Inc., "Plug-in Architectures." Accessed: Apr. 01, 2026. [Online]. Available: <https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/LoadingCode/Concepts/Plugins.html>
- [2] J. Sterne, "plug-in." Accessed: Apr. 30, 2026. [Online]. Available: <https://www.britannica.com/technology/plugin>
- [3] B. Editors, "API." Accessed: Apr. 30, 2026. [Online]. Available: <https://www.britannica.com/technology/API>
- [4] T. S. Developers, "Stabby: Defining ABI-stable types." Accessed: Apr. 07, 2026. [Online]. Available: https://docs.rs/stabby/latest/stabby/_tutorial_/index.html#defining-abi-stable-types
- [5] M. O. Manero, "Plugins in Rust: Getting our Hands Dirty." Accessed: Apr. 02, 2026. [Online]. Available: <https://nullderef.com/blog/plugin-impl/>
- [6] iddm, "Make Rust ABI stable enough to provide plugins functionality #1675." Accessed: Apr. 01, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/issues/1675>
- [7] steveklabnik, "Define a Rust ABI #600." Accessed: Apr. 01, 2026. [Online]. Available: <https://github.com/rust-lang/rfcs/issues/600>
- [8] isaac, "A Stable Modular ABI for Rust." Accessed: Apr. 01, 2026. [Online]. Available: <https://internals.rust-lang.org/t/a-stable-modular-abi-for-rust/12347>
- [9] "Crate criterion." Accessed: Apr. 06, 2026. [Online]. Available: <https://docs.rs/criterion/0.8.2/criterion/>
- [10] gungraun developers, "Crate gungraun." Accessed: Apr. 06, 2026. [Online]. Available: <https://docs.rs/gungraun/0.17.2/gungraun/>
- [11] T. R. Foundation, "Function black_box." Accessed: Apr. 06, 2026. [Online]. Available: https://doc.rust-lang.org/beta/std/hint/fn.black_box.html
- [12] T. R. P. Developers, "The Rustonomicon: Unwinding." Accessed: Feb. 24, 2026. [Online]. Available: <https://doc.rust-lang.org/nomicon/unwinding.html#unwinding>
- [13] P. LeVasseur, "What does it take to ship Rust in safety-critical?." Accessed: Apr. 14, 2026. [Online]. Available: <https://blog.rust-lang.org/2026/01/14/what-does-it-take-to-ship-rust-in-safety-critical/>
- [14] Ferrocene, "Ferrous Systems Achieves IEC 61508 (SIL 2) Certification for Rust Core Library Subset." Accessed: Apr. 14, 2026. [Online]. Available: <https://ferrous-systems.com/blog/ferrocene-libcore-news-release/>
- [15] T. rust_bridge Developers, "Transport Layer." Accessed: Apr. 30, 2026. [Online]. Available: <https://github.com/jrobhoward/rustbridge/blob/main/docs/TRANSPORT.md>
- [16] T. rust_bridge Developers, "Plugin Reload and Multiple Instances." Accessed: Apr. 30, 2026. [Online]. Available: <https://github.com/jrobhoward/rustbridge/blob/main/docs/ARCHITECTURE.md#plugin-reload-and-multiple-instances>